

Case Study -C03-AT2

Undo/Redo in Adobe Photoshop Using a Splay Tree

Bhuvanesh. S

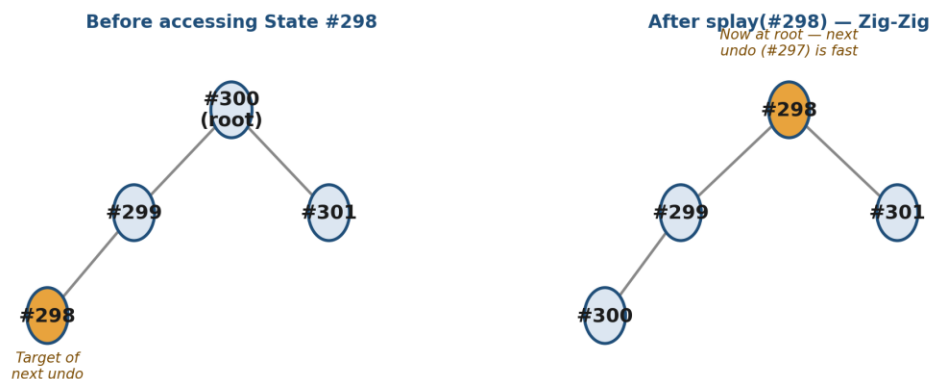
Reg. No: 192521004

1. Introduction

Adobe Photoshop keeps a history of every edit an artist makes so it can support Undo (Ctrl+Z), Redo, and jumping to any past state from the History Panel. Photoshop stores this history using a Splay Tree — a binary search tree that moves the most recently accessed node to the root using rotations. This case study explains how it works and why it fits Photoshop's usage pattern.

2. How a Splay Tree Rotation Works

Every time a history state is accessed, it is rotated up to the root through a sequence of rotations (Zig, Zig-Zig, or Zig-Zag). The example below shows a Zig-Zig rotation: state #298 is two levels deep, and one undo access brings it straight to the root.



3. Scenario A — Sequential Undo

When an artist presses Ctrl +Z repeatedly, each undo accesses the neighbouring state and splays it to the root. Because each target is already close to the top from the previous splay, repeated undo becomes very fast — amortized $O(1)$ per step.

Scenario A: Repeated Sequential Undo (Ctrl+Z x N)

Direction of repeated Undo (Ctrl+Z)

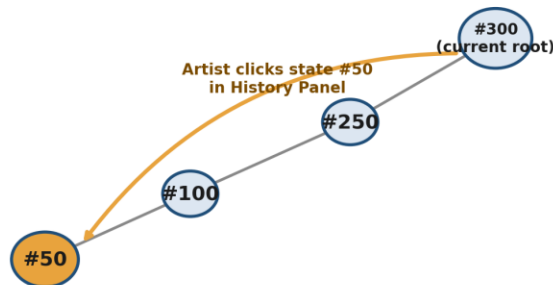


Each undo splays the previous state to the root.
Since states are neighbours, each next undo is already near the top → amortized $O(1)$ per step.

4. Scenario B — Jump via History Panel

When the artist clicks a distant state directly in the History Panel, the splay operation must restructure a long path of the tree to bring that state to the root. This is more expensive, and it also pushes the previously “warm” recent states away from the root, so the next few sequential undos are temporarily slower.

Scenario B: Jump via History Panel



Playing #50 to the root restructures the whole path (Zig-Zag chain).
States #250-#300, which were cheap to reach a moment ago, are pushed away from the root → next sequential undos are temporarily slower.

5. Why Splay Tree Beats the Alternatives Here

Structure	Best For	Fit for Undo/Redo
BST	Simple, static data	Poor — can degrade to $O(n)$
AVL Tree	Frequent lookups	Poor — no recency benefit
Red-Black Tree	Frequent insert/delete	Moderate
B-Tree	Disk-based databases	Overkill — built for disk I/O
Splay Tree	Repeated, localized access	Excellent — matches usage pattern

- Recently used states are naturally kept near the root — no other tree type does this automatically.
- No extra balance data (heights, colors) needs to be stored or maintained.
- Amortized $O(\log n)$ per operation, matching the interactive, recency-biased nature of undo/redo.

6. Conclusion

The splay tree is a strong fit for Photoshop's undo history because it optimizes automatically for the most common access pattern — repeated, sequential undo — while still bounding the occasional expensive History Panel jump at an amortized $O(\log n)$. This makes it faster in practice than a plain BST, AVL tree, Red-Black tree, or B-Tree for this specific workload.